

PARA INICIALIZAR NUTTX

1. Cargar imagen de nuttx a sd.
2. Ejecutar los siguientes comandos:
load mmc 0 0x8a000000 nuttx.bin
go 0x8a000000
3. Si la beagle está conectado a un cable ethernet, intentará conectarse pero imprimira errores.
4. Si la beagle no está conectada, se puede intentar ejecutar los programas my_server y my_udp_client con los siguientes comandos:
my_server &
my_udp_client 10.0.0.1 3001

INTENTO DE IMPLEMENTAR DRIVERS DE ETHERNET

Para iniciar, la configuración de nuttx no funcionó solamente cambiando el menuconfig. Para resolver esto, se intentó implementar drivers al inicializar el board. Para esto, se creó el archivo arm335x_cpsw.c en donde se empezó a inicializar los registros necesarios para activar las funciones de ethernet.

```
my_udp_client:0:30:20: warning: unused variable 'udph' [-Wunused-variable]
38 |         struct udphdr *udph = (struct udphdr*)(buffer + iph->ihl * 4);
    |         ^~~~~~
CPP: /home/sirk/Documents/ProyectoCC8/nuttxspace/nuttx/boards/arm/am335x/beaglebone-black/scripts/sdram.
ld-> /home/sirk/Documents/ProyectoCC8/nuttxspace/nuttx/boards/arm/am335x/beaglebone-black/scripts/sdram.l
am335x_bringup.c: In function 'am335x_bringup':
am335x_bringup.c:143:9: error: implicit declaration of function 'board_initialize'; did you mean 'umm_ini
tialize'? [-Wimplicit-function-declaration]
143 |     ret = board_initialize();
    |           ^~~~~~
    |           umm_initialize
make[2]: *** [/home/sirk/Documents/ProyectoCC8/nuttxspace/nuttx/boards/Board.mk:83: am335x_bringup.o] Err
or 1
make[1]: *** [Makefile:187: board/libboard.a] Error 2
make: *** [tools/Unix.mk:552: nuttx] Error 2
sirk@fedora:~/Documents/ProyectoCC8/nuttxspace/nuttx$ make menuconfig
```

Aquí se encontraron errores de compilación relacionados con `board\_initialize` o referencias implícitas en `am335x\_bringup.c`.

Solución: Se corrigió el archivo `am335x\_bringup.c` para llamar `am335x\_cpsw\_initialize()` correctamente, asegurando que el driver se inicialice en la fase adecuada del arranque.

Con esto, se logró la inicialización de `eth0` y el driver se registró.

```
## Starting application at 0x8A000000 ...
netdev_register: Registered MAC: 00:00:00:00:00:00 as dev: lo
lo_ifup: Bringing up: 127.0.0.1
am335x_cpsw_initialize: CPSW Initializing
am335x_cpsw_initialize: CPSW Pinmux Complete
am335x_cpsw_initialize: PHY ID: 971e971e
netdev_register: Registered MAC: 00:00:00:00:00:00 as dev: eth0
am335x_cpsw_initialize: CPSW Driver Registered
netdev_ifr_ioctl: cmd: 1813
netdev_ifr_ioctl: cmd: 1794
netdev_ifr_ioctl: cmd: 1796
netdev_ifr_ioctl: cmd: 1800
netdev_ifr_ioctl: cmd: 1818

NuttShell (NSH) NuttX-12.11.0
nsh> ls /proc/net
/proc/net:
stat
tcp
udp
lo
eth0
```

Aquí se encontró el problema de la dirección MAC que se leía como `00:00:00:00:00:00` y el PHY ID como `00000000`.

Solución: Se corrigieron las definiciones de registros MDIO y las funciones `am335x_mdio_read` y `am335x_mdio_write` en `hardware/am335x_cpsw.h` y `am335x_cpsw.c` basándose en el AM335x TRM.

El PHY ID se leyó correctamente (0007c0f1) y la dirección MAC del eFuse se obtuvo con éxito (e8:60:00:00:07:73).

```
netdev_register: Registered MAC: 00:00:00:00:00:00 as dev: lo
lo_ifup: Bringing up: 127.0.0.1
am335x_cpsw_initialize: CPSW Initializing
am335x_cpsw_initialize: CPSW Pinmux Complete
am335x_cpsw_initialize: PHY ID: 00000000
am335x_cpsw_initialize: MAC: e8:60:00:00:07:73
netdev_register: Registered MAC: e8:60:00:00:07:73 as dev: eth0
am335x_cpsw_initialize: CPSW Driver Registered
netdev_ifr_ioctl: cmd: 1813
netdev_ifr_ioctl: cmd: 1794
netdev_ifr_ioctl: cmd: 1796
netdev_ifr_ioctl: cmd: 1800
netdev_ifr_ioctl: cmd: 1818
am335x_cpsw_ifup: Bringing up eth0
lib_hostfile_lookup: ERROR: Failed to open the hosts file /etc/hosts: -2
dns_query_callback: INFO: DNS query retry 1/3
```

Una vez logrado lo anterior, nos encontramos con el siguiente error: arp_wait failed: -110. Esto nos dejó con el siguiente estado:

PHY Link: Exitoso, driver establece un enlace físico

TX Path: Funcional. Los paquetes ARP están siendo mandados.

X Path / Interrupciones: No funcional

Con esto, se pudo deducir que el board enviaba solicitudes ARP pero no recibía ni procesaba las respuestas. Esto señalaba un problema en la ruta de recepción (RX) de los paquetes.

```
am335x_cpsw_ifup: CPSW Initialized. Waiting for Link...
am335x_cpsw_ifup: Link Up! BMSR: 782d
lib_hostfile_lookup: ERROR: Failed to open the hosts file /etc/hosts: -2
dns_query_callback: INFO: DNS query retry 1/3

NuttShell (NSH) NuttX-12.11.0
nsh> arp_send: ERROR: arp_wait failed: -110, ipaddr: 10.0.0.1
arp_send: ERROR: arp_wait failed: -110, ipaddr: 10.0.0.1
arp_send: ERROR: arp_wait failed: -110, ipaddr: 10.0.0.1
arp_send: ERROR: arp_wait failed: -110, ipaddr: 10.0.0.1
arp_send: ERROR: arp_wait failed: -110, ipaddr: 10.0.0.1
psock_udp_sendto: ERROR: Not reachable
dns_send_query: ERROR: sendto failed: -101
dns_query_error: ERROR: IPv4 dns_send_query failed: -101, server address: 10.0.0.1
ntp_gethostip_multi: ERROR: getaddrinfo(0.pool.ntp.org): 1: Unknown error 1
ntpc_get_ntp_sample: ERROR: ntp_get_next_hostip() failed: 101
lib_hostfile_lookup: ERROR: Failed to open the hosts file /etc/hosts: -2
dns_query_callback: INFO: DNS query retry 1/3
psock_udp_sendto: ERROR: Not reachable
dns_send_query: ERROR: sendto failed: -101
dns_query_error: ERROR: IPv4 dns_send_query failed: -101, server address: 10.0.0.1
ntp_gethostip_multi: ERROR: getaddrinfo(1.pool.ntp.org): 1: Unknown error 1
ntpc_get_ntp_sample: ERROR: ntp_get_next_hostip() failed: 101
lib_hostfile_lookup: ERROR: Failed to open the hosts file /etc/hosts: -2
dns_query_callback: INFO: DNS query retry 1/3
psock_udp_sendto: ERROR: Not reachable
dns_send_query: ERROR: sendto failed: -101
dns_query_error: ERROR: IPv4 dns_send_query failed: -101, server address: 10.0.0.1
ntp_gethostip_multi: ERROR: getaddrinfo(2.pool.ntp.org): 1: Unknown error 1
ntpc_get_ntp_sample: ERROR: ntp_get_next_hostip() failed: 101
lib_hostfile_lookup: ERROR: Failed to open the hosts file /etc/hosts: -2
dns_query_callback: INFO: DNS query retry 1/3
psock_udp_sendto: ERROR: Not reachable
dns_send_query: ERROR: sendto failed: -101
dns_query_error: ERROR: IPv4 dns_send_query failed: -101, server address: 10.0.0.1
ntp_gethostip_multi: ERROR: getaddrinfo(0.pool.ntp.org): 1: Unknown error 1
ntpc_get_ntp_sample: ERROR: ntp_get_next_hostip() failed: 101
lib_hostfile_lookup: ERROR: Failed to open the hosts file /etc/hosts: -2
dns_query_callback: INFO: DNS query retry 1/3
psock_udp_sendto: ERROR: Not reachable
dns_send_query: ERROR: sendto failed: -101
dns_query_error: ERROR: IPv4 dns_send_query failed: -101, server address: 10.0.0.1
ntp_gethostip_multi: ERROR: getaddrinfo(1.pool.ntp.org): 1: Unknown error 1
ntpc_get_ntp_sample: ERROR: ntp_get_next_hostip() failed: 101
```

Medidas tomadas para remediar este problema:

- Revisión completa de nuttx/.config
- Implementación del polling utilizando la cola de trabajo de nuttx (HPWORK) para evitar problemas relacionas a el disparo de interrupciones.
- Corrección del DMA ya que se dedujo que los paquetes llegaban a la MAC pero no se transferían al motor DMA.

Debido a la imposibilidad de avanzar con el driver de Ethernet, se cambió el enfoque para hacer funcionar las aplicaciones del usuario (my_server y my_udp_client) utilizando raw sockets sobre la interfaz de loopback.

INTENTO DE IMPLEMENTACIÓN DE LOOPBACK

Para intentar esta implementación, se prendió la beagle sin conectarla a un cable ethernet. Una vez hecho esto, al iniciar `my_server`, el socket falló con un error indicando que la familia `AF_INET` no reconocía el tipo de socket `SOCK_RAW` por lo siguiente mensaje:

inet_setup: ERROR: Unsupported type: 3

```
NuttShell (NSH) NuttX-12.11.0
nsh> my_server &
my_server [4:100]
nsh> Server starting with protocol UDP on port 3001
inet_setup: ERROR: Unsupported type: 3
psock_socket: ERROR: socket si_setup() failed: -93
socket: ERROR: psock_socket() failed: -93
ERROR opening UDP socket: Error 93
```

Para solucionarlo se añadió código al archivo de `nuttX/net/inet/inet_sockif.c` para con un case para los `SOCK_RAW` a la función `inet_setup()`. Luego también se añadió una función para manejar este tipo `SOCK` en `inet_sendto()`.

Inmediatamente después de la solución anterior, el socket seguía fallando, pero esta vez con un valor de error numérico grande y negativo que no correspondía a un error estándar.

```
NuttShell (NSH) NuttX-12.11.0
nsh> my_server &
my_server [4:100]
nsh> Server starting with protocol UDP on port 3001
psock_socket: ERROR: socket si_setup() failed: -1979388152
socket: ERROR: psock_socket() failed: -1979388152
ERROR opening UDP socket: Error 1979388152
```

Para solucionar esto, se modificó `inet_setup()` el case para `SOCK_RAW` para asegurar un retorno de éxito explícito.

Después de que la creación del socket fue exitosa, el programa my_server crasheaba inmediatamente con un data abort como se ve en el stack dump. El registro de dirección de fallo (DFAR) mostraba 0x0000001d, una dirección de memoria inválida.

```
nsh> my_server &
my_server [4:100]
nsh> Server starting with protocol UDP on port 3001
arm_dataabort: Data abort. PC: 8a033f20 DFAR: 0000001d DFSR: 00000008
dump_assert_info: Current Version: NuttX 12.11.0 2ecd2722cd-dirty Nov 26 2025 16:13:01 arm
dump_assert_info: Assertion failed panic: at file: :0 task: my_server process: my_server 0x8a033728
up_dump_register: R0: 00000000 R1: 8a0458e8 R2: 00000020 R3: 00000000
up_dump_register: R4: 8a04f748 R5: 00000000 R6: 00000003 R7: 00000002
up_dump_register: R8: 00000011 SB: 00000000 SL: 00000000 FP: 00000000
up_dump_register: IP: 00000000 SP: 8a080e10 LR: 8a033f04 PC: 8a033f20
up_dump_register: CPSR: 4000001f
dump_stackinfo: User Stack:
dump_stackinfo: base: 0x8a080af0
dump_stackinfo: size: 00004056
dump_stackinfo: sp: 0x8a080e10
stack_dump: 0x8a080df0: 00000002 00000011 00000000 00000000 00000000 00000000 8a033f20 4000001f
stack_dump: 0x8a080e10: 00000003 8a04f748 00000003 8a01f258 00000bb9 8a080ad8 00000001 00000000
stack_dump: 0x8a080e30: 8a080ad4 8a033454 00000000 ffffffff 00000010 ffffffff 00000000 ffffffff
stack_dump: 0x8a080e50: 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff
stack_dump: 0x8a080e70: 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff
stack_dump: 0x8a080e90: 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff
stack_dump: 0x8a080eb0: 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff
stack_dump: 0x8a080ed0: 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff
stack_dump: 0x8a080ef0: 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff
stack_dump: 0x8a080f10: 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff
stack_dump: 0x8a080f30: 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff
stack_dump: 0x8a080f50: 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff
stack_dump: 0x8a080f70: 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff
stack_dump: 0x8a080f90: 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff
stack_dump: 0x8a080fb0: 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff
stack_dump: 0x8a080fd0: 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff 00000000 ffffffff
```

Para solucionarlo, se eliminaron ciertas llamadas de funciones no necesarias en el código de my_server ya que no aportaban en el uso de raw sockets.

Después de aplicar todos los parches al kernel y a las aplicaciones, se realizó una prueba final. En esta prueba al se logró ejecutar el programa de my_udp_client y se puede observar que se intenta conectar utilizando loopback pero aún se encontraron errores que no permitieron la comunicación completa.

```
NuttShell (NSH) NuttX-12.11.0
nsh> my_server &
my_server [4:100]
nsh> Server starting with protocol UDP on port 3001
inet_setup: psock=0x8a087200, type=3
inet_bind: Loopback device found: lo, ifindex: 1
UDP Server is listening on port 3001
inet_rcvmsg: DEBUG: rcvmsg psock=0x8a087200, conn=0x8a04fc14, ifindex_before=1
my_udp_client 127.0.0.1 3001
Targeting loopback server at 127.0.0.1:3001
inet_setup: psock=0x8a0885e0, type=3
ipv4_setsockopt: option: 3
ipv4_setsockopt: ipv4_setsockopt: IP_HDRINCL set (implicit for SOCK_RAW)
inet_rcvmsg: DEBUG: rcvmsg psock=0x8a0885e0, conn=0x8a04fc40, ifindex_before=0
inet_rcvmsg: WARNING: Restoring lost ifindex to 1 (Loopback)
```